Reporte

Nombres:	**Matrícula:**
Mariana Flores Rosales	**2143017**
Luis Ricardo Salinas De la Peña	**2142924**
Kenner Shahid López Ramírez	**2142291**

Nombre del curso: Analítica y inteligencia de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Fecha: 28/10/2026	**Actividad**: Laboratorio guiado

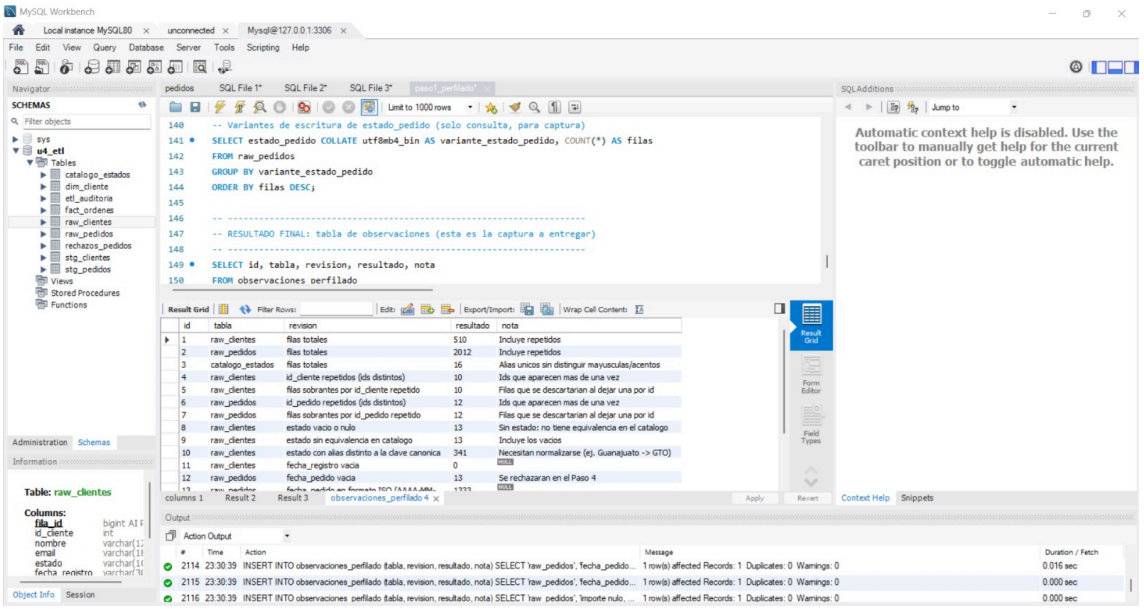
Objetivo:

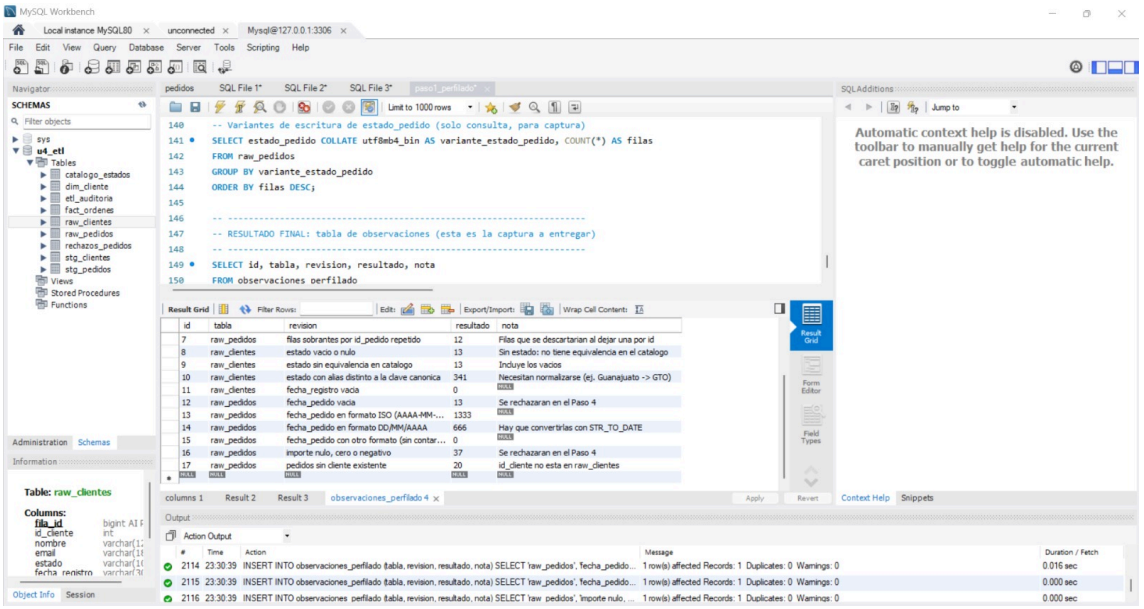
Una tienda de comercio electrónico recibe clientes.csv y pedidos.sql. El objetivo es construir un Data Mart con ventas por estado y fecha, trazabilidad de rechazos y controles reproducibles. Los datos contienen errores intencionales: no se deben corregir manualmente los archivos originales.

Entregables: script SQL propio, capturas de conteos y rechazos, consulta final de indicadores y una breve reflexión sobre decisiones de limpieza.

Contenido:

Paso 1: Perfilado





Resultados de perfilado:

Se perfilaron las tablas de origen raw_clientes, raw_pedidos y catalogo_estados, sin modificarlas.Los resultados se guardaron en la tabla observaciones_perfilado,

Columnas, claves y tipos de datos. En raw_clientes y raw_pedidos, la clave primaria es fila_id, un identificador autogenerado que no pertenece al negocio. Las claves de negocio, id_cliente e id_pedido, son INT y no tienen restricción de union, por lo que la tabla permite repetidos. Las fechas (fecha_registro y fecha_pedido) se almacenan como texto, para conservar el formato original. El importe es DECIMAL(12,2). En catalogo_estados, la clave primaria es alias.

Conteo de registros. raw_clientes tiene 510 filas, raw_pedidos tiene 2012 y catalogo_estados tiene 16 alias únicos. El catálogo trae 32 filas en el CSV, pero la clave primaria no distingue mayúsculas ni acentos, así que variantes como nuevo león y Nuevo León cuentan como el mismo alias.

Claves repetidas. Hay 10 id_cliente repetidos (10 filas sobrantes) y 12 id_pedido repetidos (12 filas sobrantes). Al revisarlos, las filas repetidas son copias idénticas.

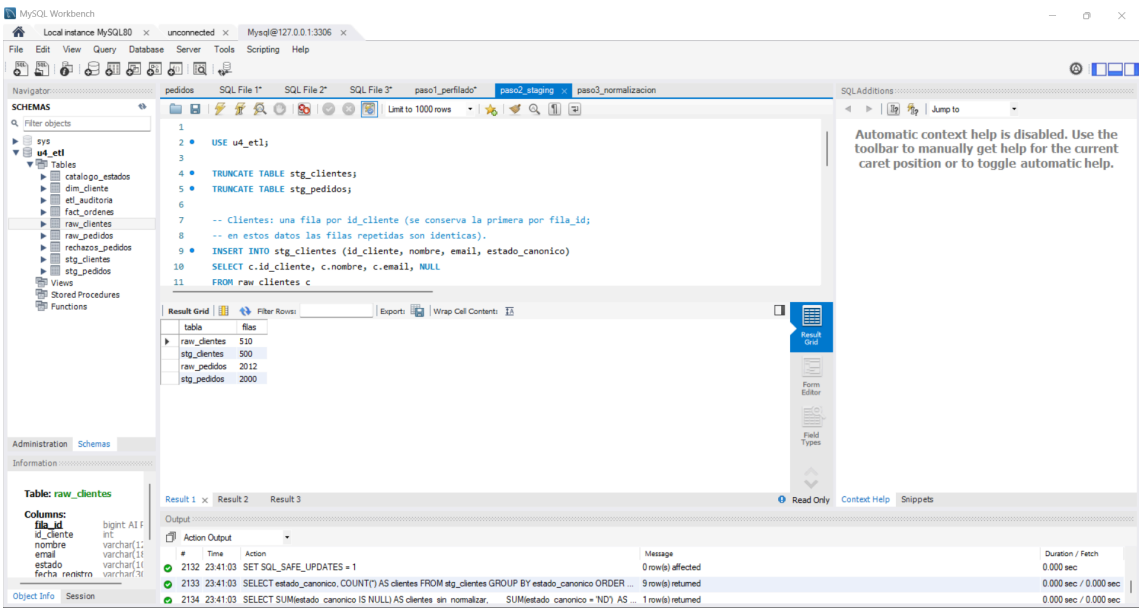
Estados no estandarizados. 13 clientes tienen el estado vacío y, por tanto, no tienen equivalencia en el catálogo. Otros 341 clientes tienen un estado válido pero escrito de forma distinta a la clave canónica (por ejemplo, Guanajuato en lugar de GTO), por lo que deben normalizarse.

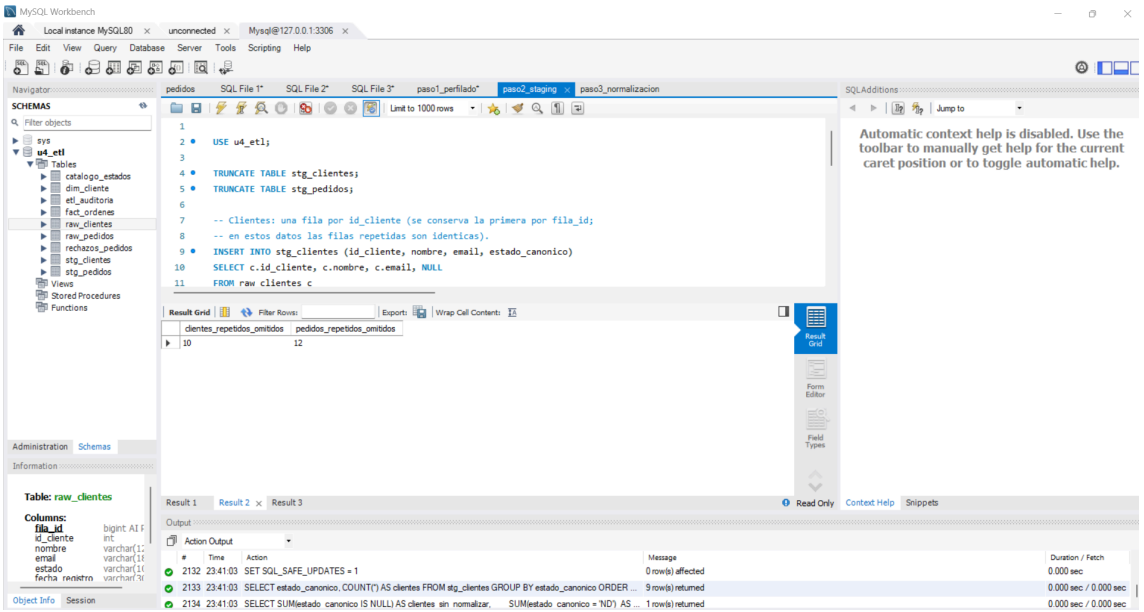
Fechas vacías. No hay fechas de registro vacías en clientes. En pedidos hay 13 fechas vacías. Además, las fechas de pedido mezclan dos formatos: 1333 en formato ISO (AAAA-MM-DD) y 666 en DD/MM/AAAA, por lo que será necesario convertirlas al tipo DATE.

Montos no positivos. 37 pedidos tienen un importe nulo, cero o negativo.

Otros hallazgos. 20 pedidos apuntan a un id_cliente que no existe en raw_clientes. Además, estado_pedido aparece escrito de varias formas (entregado, ENTREGADO, Entregado).

Paso 2: Staging





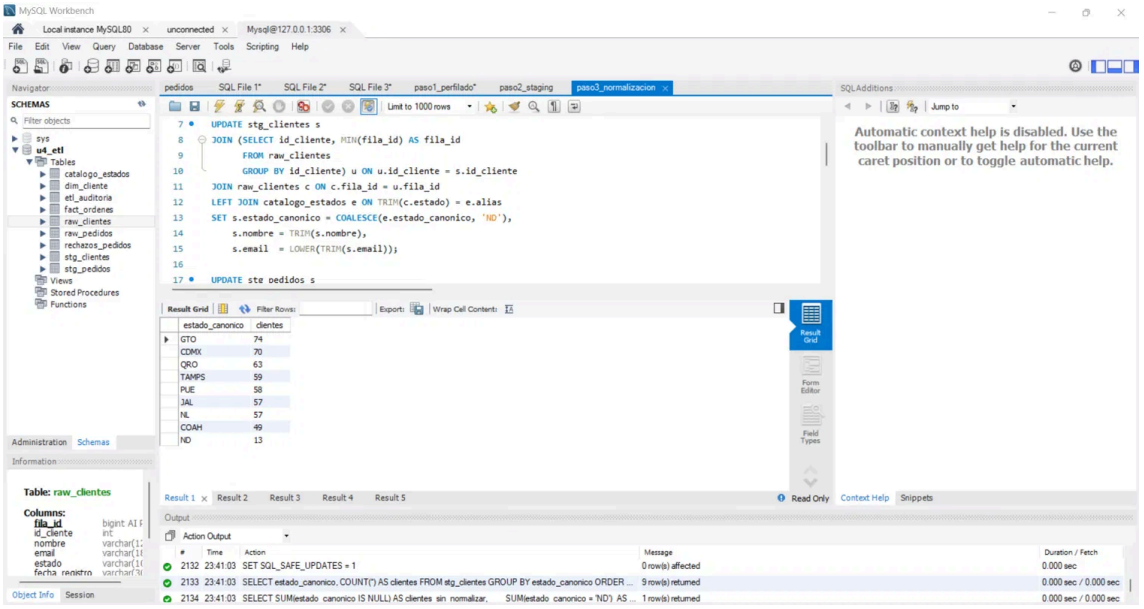
Estructura de zonas. Se utilizó raw_* como zona de aterrizaje inmutable y stg_* como zona de datos normalizados. Los datos se copiaron de raw_clientes y raw_pedidos hacia stg_clientes y stg_pedidos, sin modificar las tablas de origen.

Resultado de la carga. raw_clientes tenía 510 filas y stg_clientes quedó con 500, es decir, se omitieron 10 filas repetidas. raw_pedidos tenía 2012 filas y stg_pedidos quedó con 2000, con 12 filas repetidas omitidas. Para cada id_cliente e id_pedido repetido se conservó la primera fila según fila_id. Se verificó que las filas repetidas eran copias idénticas, así que no se pierde información. La comprobación final muestra 0 repetidos en ambas tablas de staging, y las tablas raw_* conservan sus 510 y 2012 filas originales.

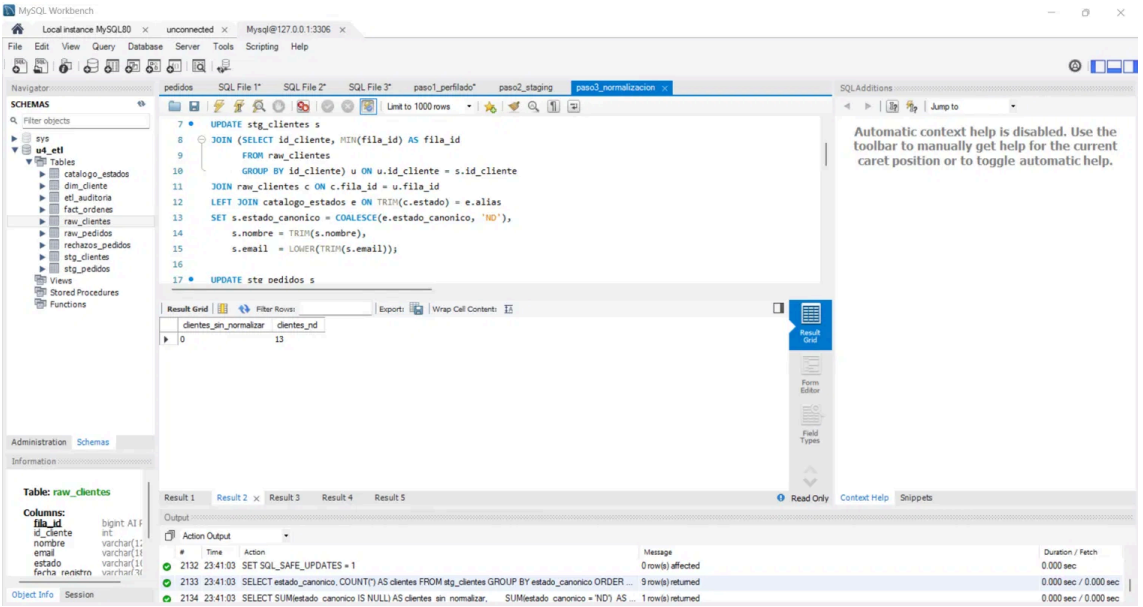
¿Por qué no se debe modificar la información original?

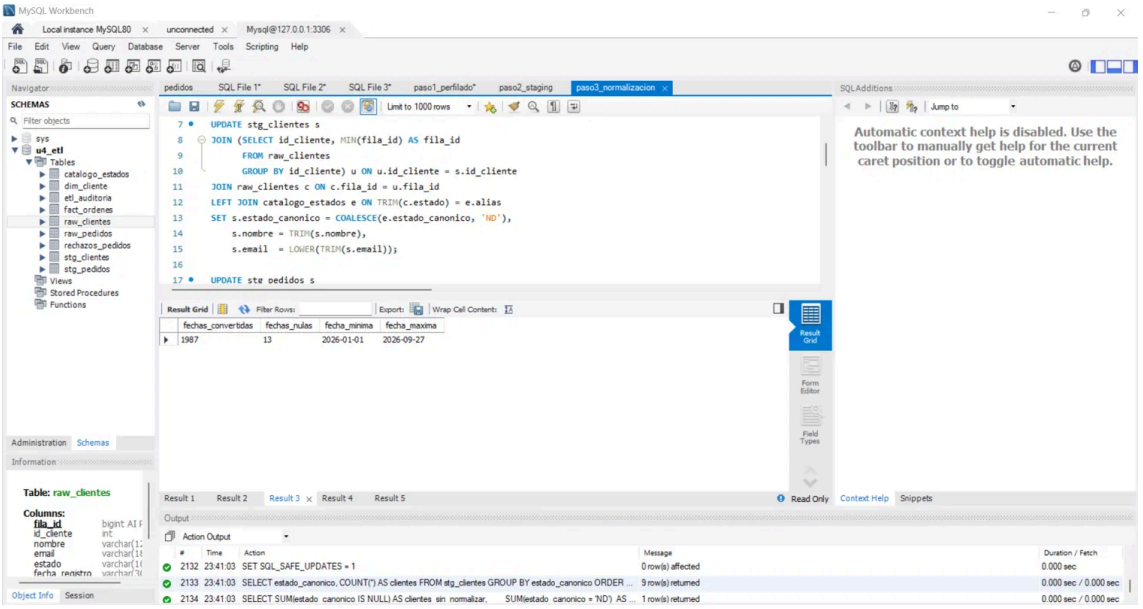
1. Trazabilidad: raw_* es la evidencia de lo que realmente llegó. Si se corrige ahí, ya no se puede demostrar qué dato venía mal ni cuántos errores tenía el origen.
2. Reproducibilidad: si una regla de limpieza falla o cambia, se vacía stg_* y se vuelve a procesar desde raw_*, sin necesidad de pedir de nuevo los archivos.
3. Auditoría y conciliación: los conteos entre origen, staging y destino solo cuadran si el origen permanece intacto.
4. Seguridad:un UPDATE o DELETE equivocado sobre el origen no se puede deshacer, mientras que en staging basta con repetir la carga.

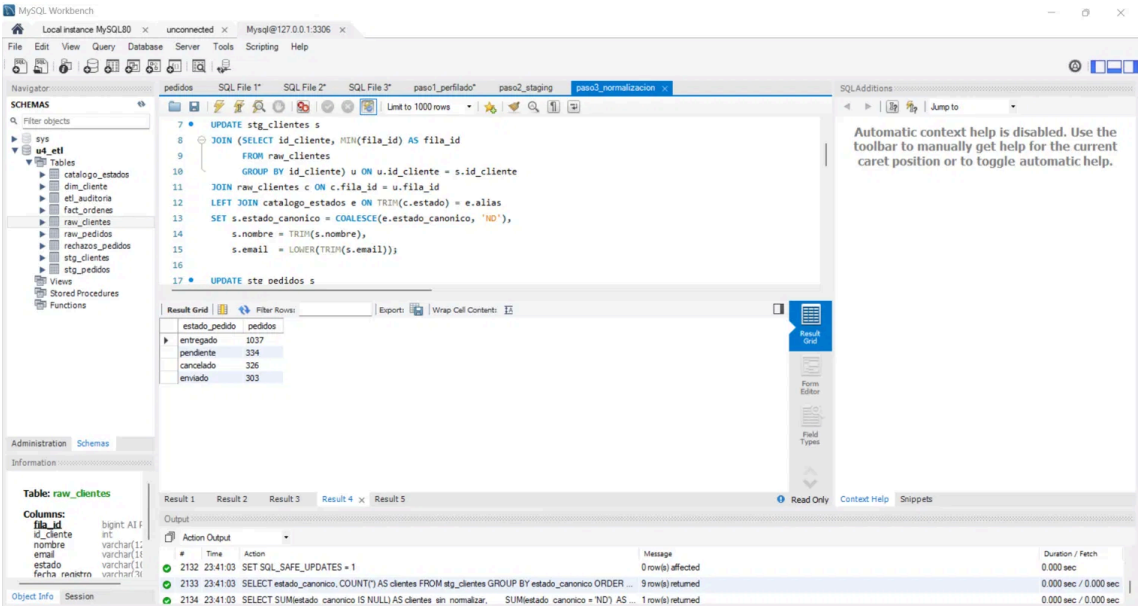
Paso 3: Normalización

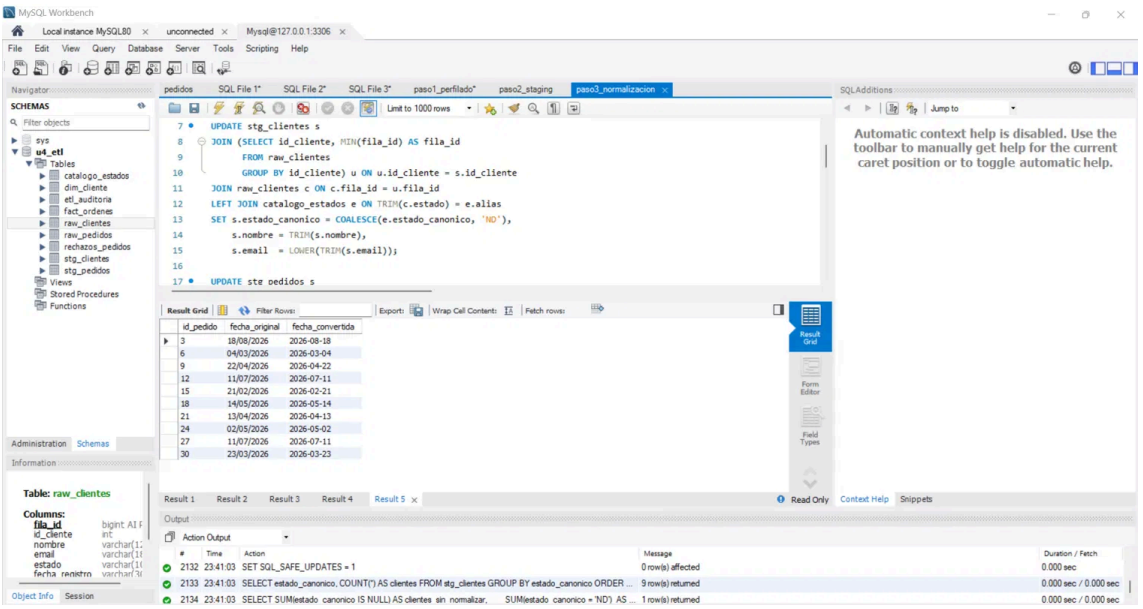


Reporte


MySQL Workbench
Local instance MySQL80
unconnected
Mysql@127.0.0.1:3306
File Edit View Query Database Server Tools Scripting Help
pedidos
SQL File 1*
SQL File 2*
SQL File 3*
paso1_perfilado*
paso2_staging
paso3_normalizacion
Limit to 1000 rows
SCHEMAS
sys
u4_etl
Tables
catalogo_estados
dim_cliente
etl_auditoria
fact_ordenes
raw_clientes
raw_pedidos
rechazos_pedidos
stg_clientes
stg_pedidos
Views
Stored Procedures
Functions
7 UPDATE stg_clientes s
8 JOIN (SELECT id_cliente, MIN(fila_id) AS fila_id
9 FROM raw_clientes
10 GROUP BY id_cliente) u ON u.id_cliente = s.id_cliente
11 JOIN raw_clientes c ON c.fila_id = u.fila_id
12 LEFT JOIN catalogo_estados e ON TRIM(c.estado) = e.alias
13 SET s.estado_canonico = COALESCE(e.estado_canonico, 'ND'),
14 s.nombre = TRIM(s.nombre),
15 s.email = LOWER(TRIM(s.email));
16
17 UPDATE stg_pedidos s
Result Grid
clientes_sin_normalizar
clientes_nd
0
13
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Result 1
Result 2
Result 3
Result 4
Result 5
Read Only
Context Help
Snippets
Output
Action Output
Time Action Message Duration / Fetch
2132 23:41:03 SET SQL_SAFE_UPDATES = 1 0 row(s) affected 0.000 sec
2133 23:41:03 SELECT estado_canonico, COUNT(*) AS clientes FROM stg_clientes GROUP BY estado_canonico ORDER ... 9 row(s) returned 0.000 sec / 0.000 sec
2134 23:41:03 SELECT SUM(estado_canonico IS NULL) AS clientes_sin_normalizar, SUM(estado_canonico = 'ND') AS ... 1 row(s) returned 0.000 sec / 0.000 sec
Table: raw_clientes
Columns:
fila_id bigint AI P
id_cliente int
nombre varchar(1
email varchar(1
estado varchar(1
fecha_registro varchar(3
Object Info
Session


MySQL Workbench
Local instance MySQL80
unconnected
Mysql@127.0.0.1:3306
File Edit View Query Database Server Tools Scripting Help
pedidos
SQL File 1*
SQL File 2*
SQL File 3*
paso1_perfilado*
paso2_staging
paso3_normalizacion
Limit to 1000 rows
SCHEMAS
sys
u4_etl
Tables
catalogo_estados
dim_cliente
etl_auditoria
fact_ordenes
raw_clientes
raw_pedidos
rechazos_pedidos
stg_clientes
stg_pedidos
Views
Stored Procedures
Functions
7 UPDATE stg_clientes s
8 JOIN (SELECT id_cliente, MIN(fila_id) AS fila_id
9 FROM raw_clientes
10 GROUP BY id_cliente) u ON u.id_cliente = s.id_cliente
11 JOIN raw_clientes c ON c.fila_id = u.fila_id
12 LEFT JOIN catalogo_estados e ON TRIM(c.estado) = e.alias
13 SET s.estado_canonico = COALESCE(e.estado_canonico, 'ND'),
14 s.nombre = TRIM(s.nombre),
15 s.email = LOWER(TRIM(s.email));
16
17 UPDATE stg_pedidos s
Result Grid
fechas_convertidas
fechas_nulas
fecha_minima
fecha_maxima
1987
13
2026-01-01
2026-09-27
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Result 1
Result 2
Result 3
Result 4
Result 5
Read Only
Context Help
Snippets
Output
Action Output
Time Action Message Duration / Fetch
2132 23:41:03 SET SQL_SAFE_UPDATES = 1 0 row(s) affected 0.000 sec
2133 23:41:03 SELECT estado_canonico, COUNT(*) AS clientes FROM stg_clientes GROUP BY estado_canonico ORDER ... 9 row(s) returned 0.000 sec / 0.000 sec
2134 23:41:03 SELECT SUM(estado_canonico IS NULL) AS clientes_sin_normalizar, SUM(estado_canonico = 'ND') AS ... 1 row(s) returned 0.000 sec / 0.000 sec
Table: raw_clientes
Columns:
fila_id bigint AI P
id_cliente int
nombre varchar(1
email varchar(1
estado varchar(1
fecha_registro varchar(3
Object Info
Session





Estados. Se utilizó catalogo_estados para convertir nombres y abreviaturas en una clave estatal uniforme. Por ejemplo, Guanajuato, GUANAJUATO y guanajuato quedaron como GTO, y CIUDAD DE MÉXICO quedó como CDMX. El resultado fue de 487 clientes con estado reconocido, distribuidos en 8 estados (GTO 74, CDMX 70, QRO 63, TAMPS 59, PUE 58, JAL 57, NL 57 y COAH 49), más 13 clientes con la clave ND. La verificación muestra 0 clientes sin normalizar.

Fechas. Las fechas en formato ISO (AAAA-MM-DD) y DD/MM/AAAA se convirtieron al tipo DATE. De los 2000 pedidos, 1987 se convirtieron correctamente y 13 quedaron en NULL porque venían vacíos. Las fechas van del 2026-01-01 al 2026-09-27. Que todas

caigan en 2026 confirma que el día y el mes no se interpretaron al revés. Por ejemplo, 18/08/2026 pasó a 2026-08-18 y 04/03/2026 a 2026-03-04.

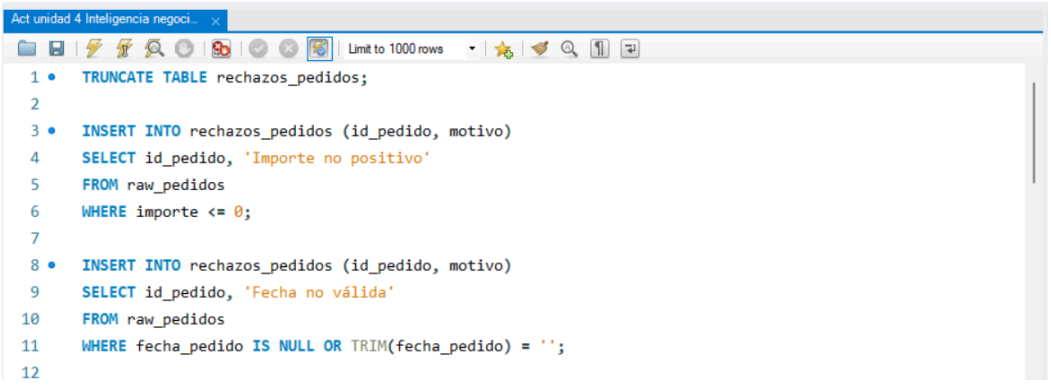
Otras normalizaciones. El estado_pedido se estandarizó a minúsculas y sin espacios sobrantes. Las variantes entregado, ENTREGADO y Entregado se unificaron en un solo valor. El resultado fue de 1037 pedidos entregados, 334 pendientes, 326 cancelados y 303 enviados. También se limpiaron los espacios del nombre y se pasó el email a minúsculas.

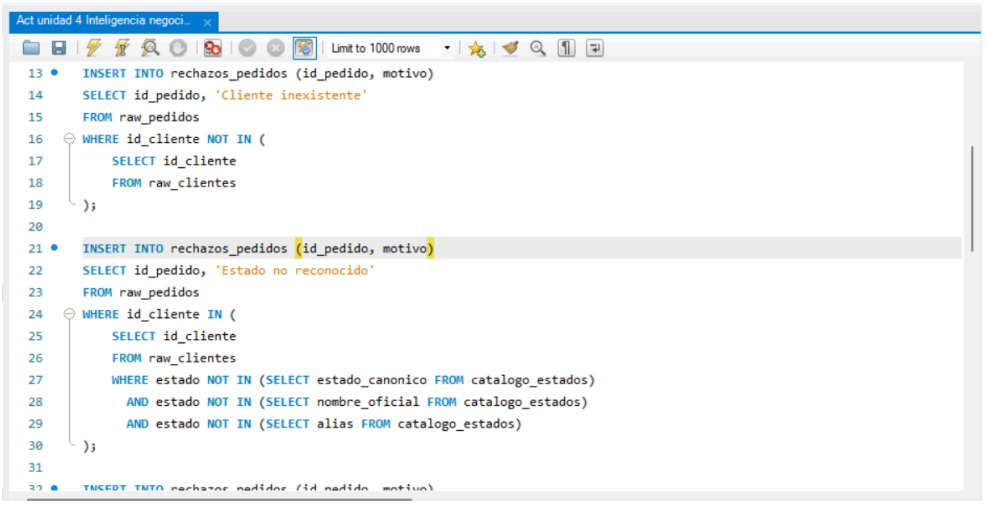
¿Cómo trataría los estados sin equivalencia?

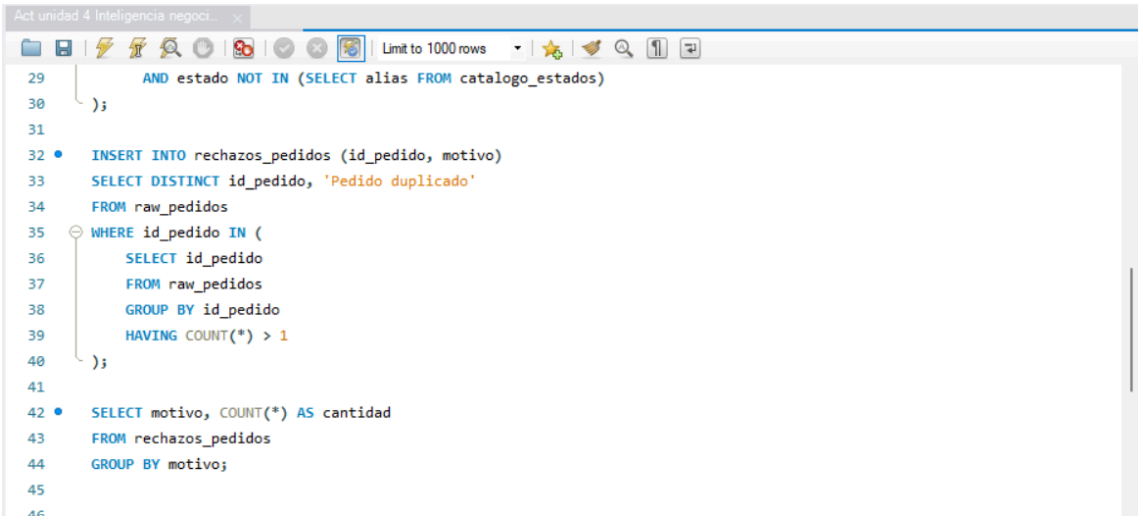
No se inventa ni se adivina un estado.

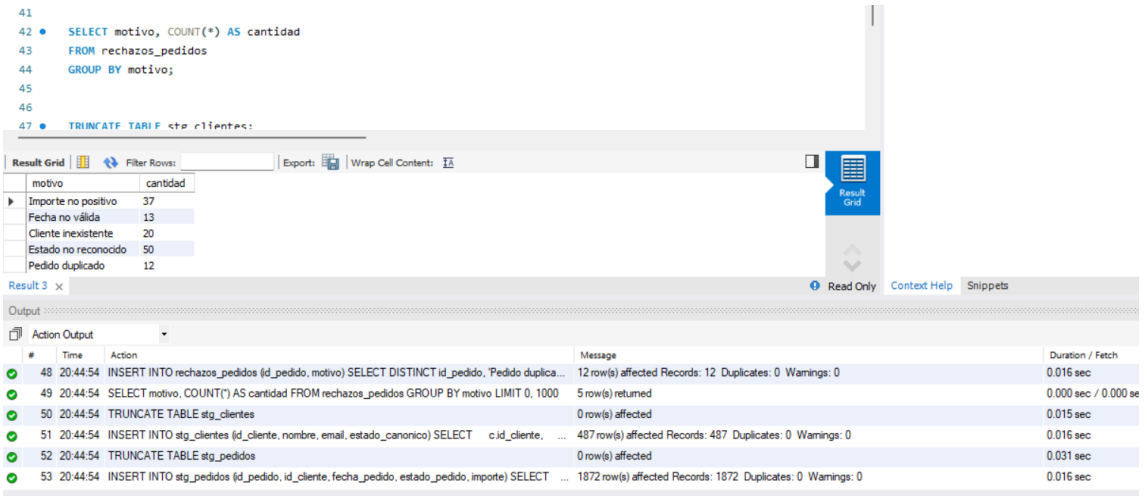
- Se asigna una clave explícita, ND (no identificado), y el cliente se conserva, para no perder sus pedidos y sus ventas.
- Se registra cuántos casos hay (13) y qué valor traían, para que el responsable del catálogo decida si se agrega un alias nuevo.
- Si se agrega el alias, basta con volver a ejecutar la normalización desde raw_*, porque el origen nunca se alteró.

Paso 4: Validación y Rechazos









Resultados de la validación:

Se aplicaron 5 reglas de calidad para identificar anomalías en los datos crudos. Las filas con errores se registraron en la tabla rechazos_pedidos como bitácora de auditoría, dejando intacto el origen.

- **Importes no positivos**. Se detectaron 37 pedidos con un importe nulo, menor o igual a cero.
- **Fechas no válidas**. Se encontraron 13 pedidos con fecha nula o vacía.
- **Clientes inexistentes**. Existen 20 pedidos que apuntan a un id_cliente que no está registrado en raw_clientes (pedidos huérfanos).
- **Estados no reconocidos**. Se rechazaron 50 pedidos porque la ubicación del cliente no coincide con ningún estado_canonico, nombre_oficial o alias dentro del catalogo_estados.
- **Pedidos duplicados**. Se detectaron 12 IDs de pedido duplicados en raw_pedidos.

En total, la bitácora capturó **132 rechazos**. Se utilizó TRUNCATE TABLE antes de las inserciones para asegurar que el script sea reproducible y sin duplicidad por múltiples ejecuciones.

Paso 5: Carga final de Staging (Datos Limpios)

```
46
47 •  TRUNCATE TABLE stg_clientes;
48
49 •  INSERT INTO stg_clientes (id_cliente, nombre, email, estado_canonico)
50    SELECT
51        c.id_cliente,
52        c.nombre,
53        c.email,
54        MAX(cat.estado_canonico) AS estado_canonico
55    FROM raw_clientes c
56    JOIN catalogo_estados cat
57      ON c.estado = cat.estado_canonico
58      OR c.estado = cat.nombre_oficial
59      OR c.estado = cat.alias
60    GROUP BY c.id_cliente, c.nombre, c.email;
61
```

```
63 •  TRUNCATE TABLE stg_pedidos;
64
65 •  INSERT INTO stg_pedidos (id_pedido, id_cliente, fecha_pedido, estado_pedido, importe)
66    SELECT
67        id_pedido,
68        id_cliente,
69        CASE
70            WHEN fecha_pedido LIKE '%/%' THEN STR_TO_DATE(fecha_pedido, '%d/%m/%Y')
71            ELSE fecha_pedido
72        END AS fecha_pedido,
73        estado_pedido,
74        importe
75    FROM raw_pedidos
76    WHERE id_pedido NOT IN (
77        SELECT id_pedido
78        FROM rechazos_pedidos
79    );
```

```
81 •  SELECT COUNT(*) AS total_clientes_limpios FROM stg_clientes;
82 •  SELECT COUNT(*) AS total_pedidos_limpios FROM stg_pedidos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_clientes_limpios
487

Result 4 | Read Only | Context Help | Snippets

Output

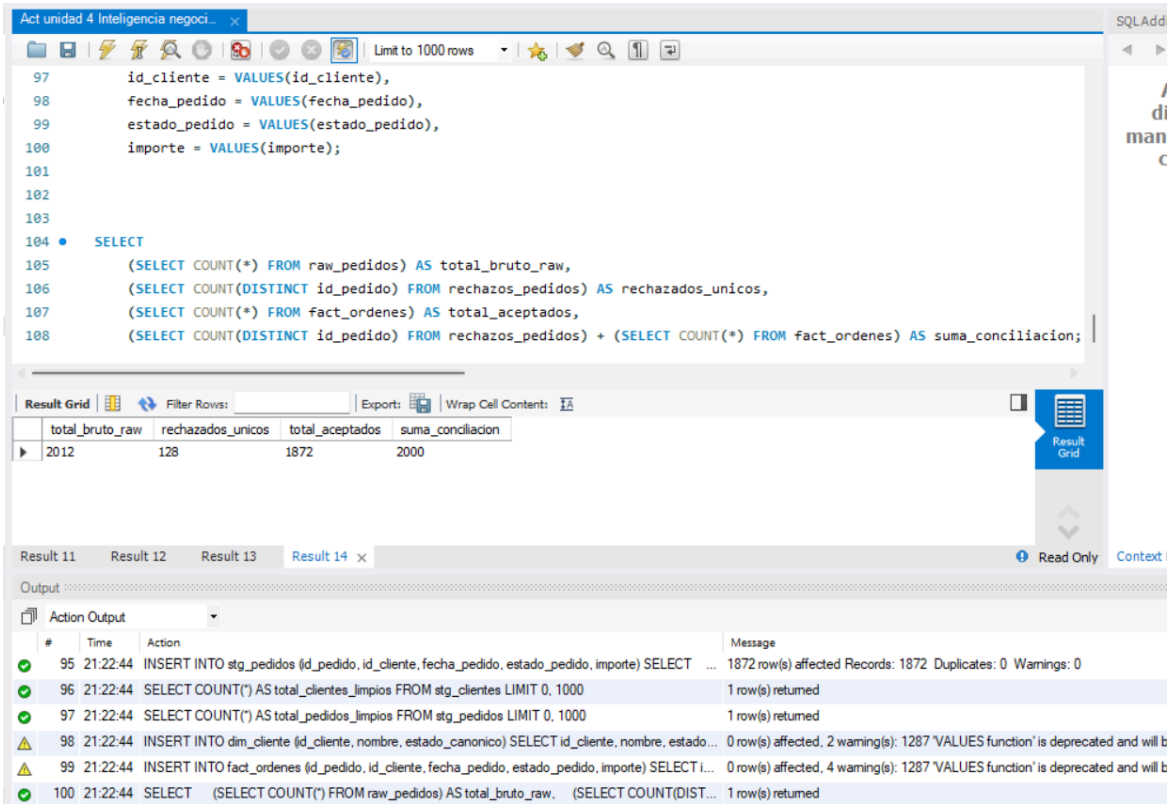
Action Output

#	Time	Action	Message	Duration / Fetch
49	20:44:54	SELECT motivo, COUNT(*) AS cantidad FROM rechazos_pedidos GROUP BY motivo LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 se
50	20:44:54	TRUNCATE TABLE stg_clientes	0 row(s) affected	0.015 sec
51	20:44:54	INSERT INTO stg_clientes (id_cliente, nombre, email, estado_canonico) SELECT c.id_cliente, ...	487 row(s) affected Records: 487 Duplicates: 0 Warnings: 0	0.016 sec
52	20:44:54	TRUNCATE TABLE stg_pedidos	0 row(s) affected	0.031 sec
53	20:44:54	INSERT INTO stg_pedidos (id_pedido, id_cliente, fecha_pedido, estado_pedido, importe) SELECT ...	1872 row(s) affected Records: 1872 Duplicates: 0 Warnings: 0	0.016 sec
54	20:48:27	SELECT COUNT(*) AS total_clientes_limpios FROM stg_clientes LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 se

Resultados de la carga:

- **Homologación de clientes**. Se pobló stg_clientes cruzando el estado original con catalogo_estados. Para evitar errores de llave duplicada (Error 1062) por alias redundantes en el catálogo, se agruparon los registros asegurando un único estado_canonico por cliente.
- **Filtrado de pedidos**. Se pobló stg_pedidos extrayendo los datos de raw_pedidos pero excluyendo estrictamente todos los registros que figuraban en rechazos_pedidos (mediante NOT IN). De esta forma, la capa staging final aloja únicamente datos limpios y estandarizados, listos para la creación del Data Mart.

Paso 6: Conciliación y Auditoría



Justificación de conciliación:

Al realizar la conciliación final, la suma de los pedidos aceptados (1,872) y los IDs únicos rechazados (128) arroja un resultado de 2,000. Esto presenta una diferencia de 12 registros respecto al conteo bruto original de raw_pedidos (2,012).

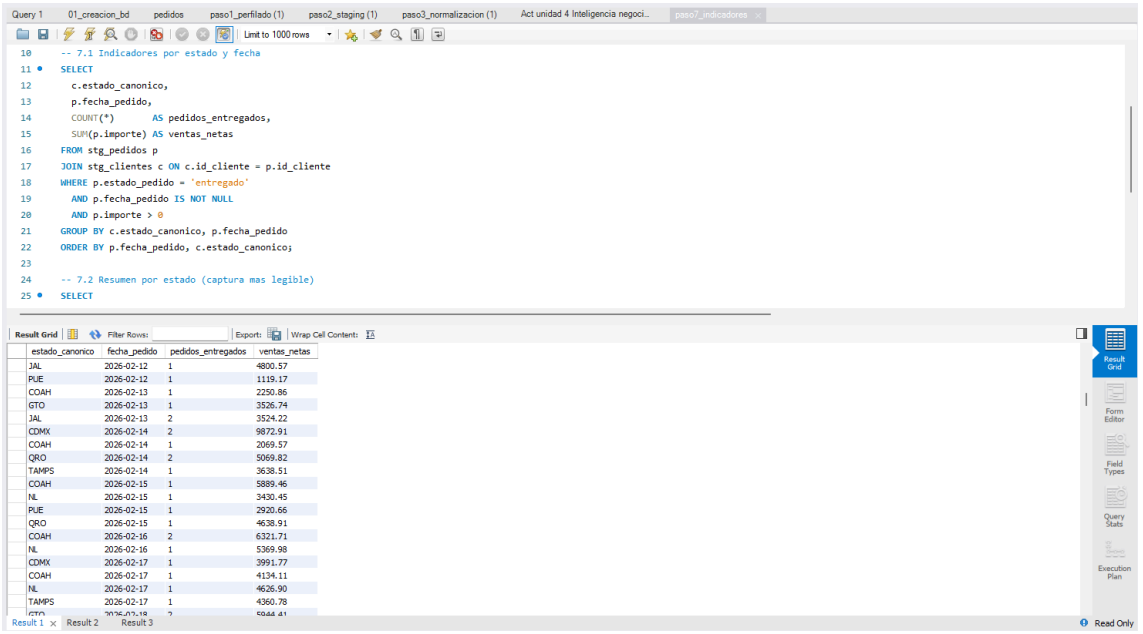
Esta aparente diferencia se justifica matemáticamente por la naturaleza de los datos crudos:

- **Pedidos con múltiples errores**: Aunque la bitácora registró 132 fallos en las reglas de calidad, estos corresponden a 128 IDs únicos de pedido (4 pedidos incumplieron más de una regla simultáneamente).
- **Impacto de los duplicados**: De esos 128 IDs únicos rechazados, 12 corresponden a IDs que venían duplicados desde el origen (lo que representa 24 filas físicas en la tabla cruda).

- **Balance final**: Al filtrar mediante NOT IN los 128 IDs problemáticos, el proceso ETL descartó en la práctica 140 filas físicas en total (116 sencillas + 24 duplicadas).

Por lo tanto, la ecuación final correcta de auditoría es: 1,872 filas aceptadas + 140 filas físicas eliminadas = 2,012 filas crudas totales. Con esto se certifica la exactitud del modelo Data Mart, la integridad relacional y se garantiza que no hubo pérdida de información injustificada durante la carga incremental (UPSERT).

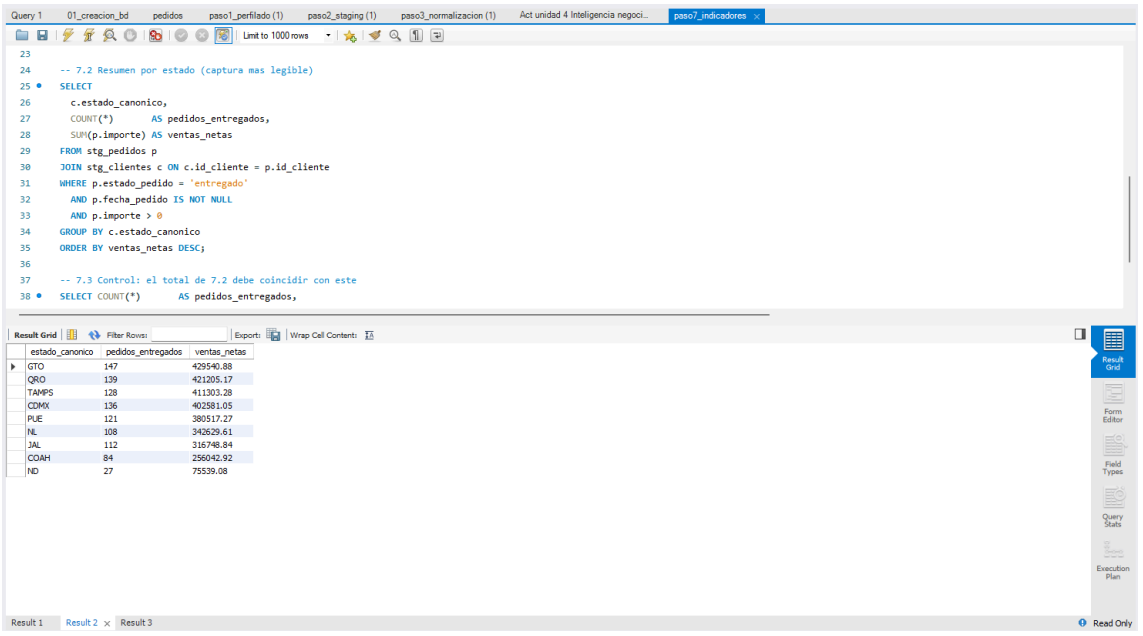
Paso 7: Indicadores



```
-- 7.1 Indicadores por estado y fecha
SELECT
  c.estado_canonico,
  p.fecha_pedido,
  COUNT(*)        AS pedidos_entregados,
  SUM(p.importe) AS ventas_netas
FROM stg_pedidos p
JOIN stg_clientes c ON c.id_cliente = p.id_cliente
WHERE p.estado_pedido = 'entregado'
  AND p.fecha_pedido IS NOT NULL
  AND p.importe > 0
GROUP BY c.estado_canonico, p.fecha_pedido
ORDER BY p.fecha_pedido, c.estado_canonico;

-- 7.2 Resumen por estado (captura mas legible)
SELECT
```

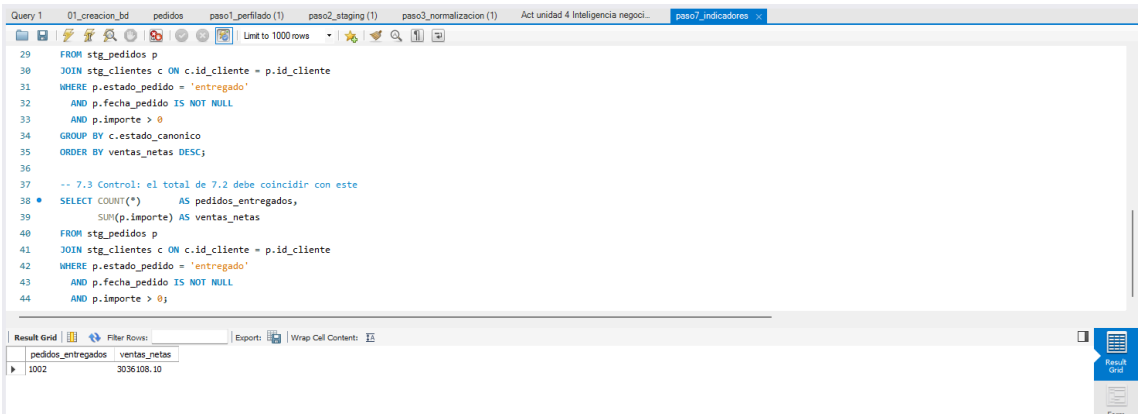
estado_canonico	fecha_pedido	pedidos_entregados	ventas_netas
JAL	2026-02-12	1	4800.57
PUE	2026-02-12	1	1119.17
COAH	2026-02-13	1	2250.86
GTO	2026-02-13	1	3526.74
JAL	2026-02-13	2	3524.22
CDMX	2026-02-14	2	9872.91
COAH	2026-02-14	1	2069.57
QRO	2026-02-14	2	5069.82
TAMPS	2026-02-14	1	3638.51
COAH	2026-02-15	1	5889.46
NL	2026-02-15	1	3430.45
PUE	2026-02-15	1	2920.66
QRO	2026-02-15	1	4638.91
COAH	2026-02-16	2	6321.71
NL	2026-02-16	1	5369.98
CDMX	2026-02-17	1	3991.77
COAH	2026-02-17	1	4134.11
NL	2026-02-17	1	4626.90
TAMPS	2026-02-17	1	4360.78



```
-- 7.2 Resumen por estado (captura mas legible)
SELECT
  c.estado_canonico,
  COUNT(*)        AS pedidos_entregados,
  SUM(p.importe) AS ventas_netas
FROM stg_pedidos p
JOIN stg_clientes c ON c.id_cliente = p.id_cliente
WHERE p.estado_pedido = 'entregado'
  AND p.fecha_pedido IS NOT NULL
  AND p.importe > 0
GROUP BY c.estado_canonico
ORDER BY ventas_netas DESC;

-- 7.3 Control: el total de 7.2 debe coincidir con este
SELECT COUNT(*)        AS pedidos_entregados,
```

estado_canonico	pedidos_entregados	ventas_netas
GTO	147	429540.88
QRO	139	421205.17
TAMPS	128	411303.28
CDMX	136	402581.05
PUE	121	380517.27
NL	108	342629.61
JAL	112	316748.84
COAH	84	256042.92
ND	27	75539.08



Filtros del Paso 7

- JOIN con stg_clientes: el estado está en el cliente, no en el pedido. Al ser un JOIN interno, también descarta pedidos de clientes inexistentes.
- estado_pedido = 'entregado': por definición del laboratorio, solo los pedidos entregados cuentan como venta.
- fecha_pedido IS NOT NULL: un pedido sin fecha no se puede ubicar en un día.
- importe > 0: un importe nulo, cero o negativo no es una venta válida.
- GROUP BY: en 7.1 agrupa por estado y fecha; en 7.2 solo por estado, para ver el total de cada uno.
- 7.3: mismos filtros sin agrupar, como control. Su total debe coincidir con la suma de 7.2.

El total es menor a los 1037 entregados del Paso 3 porque aquí se excluyen los pedidos que no cumplen las reglas de calidad.

Supuestos del modelo:

Para este laboratorio, las ventas netas se definen como la suma del importe de los pedidos con estado entregado. No se modelan descuentos, impuestos, costos de envío ni devoluciones, por lo que esta cifra no equivale a una venta neta contable real. Solo se consideran pedidos válidos: con cliente existente, fecha válida e importe mayor a cero. Los clientes sin estado identificable se agrupan bajo la clave ND en lugar de rechazarse, para conservar sus ventas.

Preguntas de reflexión

¿Qué ocurre si un cliente tiene estado vacío? ¿Se rechaza el cliente o se agrupa como no identificado?

Se agrupa como no identificado (ND) y no se rechaza. El estado vacío es un defecto del dato del cliente, no del pedido, y rechazarlo haría perder ventas reales y válidas. Con ND el problema queda visible y medible: se sabe cuántos clientes son (13) y cuánto venden. El costo es que esas ventas no se pueden analizar por estado,

así que se reportan aparte. Si luego el responsable del catálogo corrige el dato, basta con volver a ejecutar la normalización desde raw_*.

¿Cómo detectarías pedidos que llegaron tarde?

Agregaría a la tabla de hechos una columna con la fecha en que cada pedido se cargó al Data Mart (fecha_carga) y la compararía con fecha_pedido. Si la diferencia supera un umbral definido (por ejemplo, 2 días), se marca con una bandera llegada_tardia. Como la carga es incremental, también se puede detectar cuando llega un pedido con fecha anterior al último lote ya cargado. Esos casos conviene revisarlos, porque modifican totales de días ya reportados.

¿Cómo modificarías el proceso para admitir devoluciones?

Crearía una tabla fact_devoluciones (id_devolucion, id_pedido, fecha_devolucion, importe) ligada al pedido original. Pasaría por las mismas validaciones: pedido existente, fecha válida e importe mayor a cero, y además que lo devuelto no supere lo vendido. La carga sería con UPSERT por id_devolucion, para mantener la idempotencia. Las ventas netas pasarían a definirse como la suma de pedidos entregados menos las devoluciones, y se agregaría una columna de devoluciones al indicador por estado y fecha.

Link al repositorio:

https://github.com/KSinloop/Act-04-Inteligencia-de-Negocios.git